

LAPORAN PRAKTIKUM 8

SORTING ALGORITHM

STRUKTUR DATA



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Sorting Algorithm” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Laporan ini akan membahas secara mendalam mengenai konsep, logika dasar, serta implementasi berbagai algoritma pengurutan data (Sorting) menggunakan bahasa pemrograman Java. Fokus utama pembahasan mencakup tiga algoritma yaitu *Shell Sort*, *Bubble Sort*, *Quick Sort*, dan *Merge Sort*, serta implementasi visualisasinya menggunakan GUI Java Swing. Diharapkan melalui pembahasan ini, pemahaman mengenai bagaimana data dikelola, diurutkan, dan divisualisasikan secara efisien dalam memori komputer dapat tergambar dengan jelas.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dari segi penulisan maupun analisis kode program. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 2 Juni 2026

Amiratul Fadhillah

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN	
2.1 Landasan Teori	3
2.2 Alat dan Bahan.....	3
2.3 Langkah Praktikum dan Kode Program	4
2.3.1 Kelas Shell Sort.....	4
2.3.2 Kelas Quick Sort.....	5
2.3.3 Kelas Merge Sort	6
2.3.4 Kelas Bubble Sort GUI.....	7
2.3.5 Kelas Merge Sort GUI.....	7
BAB III PENUTUP	
3.1 Kesimpulan.....	13
3.2 Saran.....	13
DAFTAR PUSTAKA.....	14
TUGAS 8 PRAKTIKUM STRUKTUR DATA.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ranah ilmu komputer dan rekayasa perangkat lunak, struktur data dan algoritma merupakan dua pilar utama yang sangat krusial dalam menentukan efisiensi suatu program. Mengelola himpunan data yang berskala besar membutuhkan teknik pengurutan (*sorting*) agar proses pencarian, penyisipan, dan manipulasi informasi di dalamnya dapat berjalan secara optimal. Tanpa adanya algoritma pengurutan yang tepat, komputer akan mengonsumsi waktu komputasi dan memori yang jauh lebih besar, yang pada akhirnya menurunkan performa sistem secara keseluruhan.

Terdapat berbagai macam pendekatan algoritma pengurutan, di antaranya adalah *Shell Sort*, *Quick Sort*, *Merge Sort*, dan *Bubble Sort*, yang masing-masing dirancang dengan karakteristik kompleksitas waktu dan ruang yang beraneka ragam. Pemilihan algoritma ini biasanya didasarkan pada skala data serta kondisi himpunan data yang akan diproses. Melalui kegiatan praktikum ini, setiap konsep teoretis dari algoritma tersebut tidak hanya diuji melalui antarmuka konsol standar, melainkan juga divisualisasikan secara dinamis menggunakan *Graphical User Interface* (GUI). Visualisasi interaktif ini sangat penting untuk menjembatani komunikasi data dan menyimulasikan cara kerja internal memori, sebuah konsep krusial yang juga sangat bermanfaat ketika informasi teknis perlu disajikan atau disosialisasikan kepada audiens secara luas.

1.2 Tujuan Praktikum

- 1) Mampu merancang dan mengimplementasikan algoritma pengurutan data (*Shell Sort*, *Quick Sort*, *Merge Sort*, dan *Bubble Sort*) ke dalam sintaks bahasa pemrograman Java.
- 2) Memahami perbedaan fundamental, tingkat efisiensi, dan mekanisme operasional dari masing-masing metode pengurutan.
- 3) Mampu membangun program antarmuka visual (GUI) interaktif menggunakan Java Swing guna menyimulasikan algoritma pengurutan secara langkah demi langkah (*step-by-step*).

1.3 Manfaat Praktikum

Melalui praktikum ini, praktikan mendapatkan pengalaman konkret dalam memanipulasi tipe data *array* bersarang dan mengelola penunjuk indeks (pointer) di dalam struktur data. Hal ini melatih daya penalaran logis serta kepekaan terhadap *debugging*, keterampilan yang sangat mutlak diperlukan oleh pengembang perangkat lunak profesional.

Selain penguasaan algoritma, praktikum yang terintegrasi dengan GUI juga menumbuhkan kemampuan perancangan *User Interface* (UI). Praktikan dapat mengamati secara langsung perilaku data saat diubah posisinya, yang memperdalam intuisi terhadap kompleksitas waktu komputasi, serta melatih kedisiplinan pengarsipan laporan teknis secara rapi dan sistematis.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Pengurutan data (*sorting*) adalah proses pengaturan sekumpulan objek menurut urutan atau susunan tertentu yang berpedoman pada kaidah perbandingan. Berikut adalah penjelasan sederhana dari algoritma yang digunakan dalam praktikum ini:

- Shell Sort: Merupakan bentuk pengembangan dari *Insertion Sort* yang dirancang untuk mengatasi kelemahan pergeseran data pada elemen yang berjauhan. Algoritma ini menggunakan variabel interval atau *gap* untuk membandingkan dan menukar elemen, di mana nilai *gap* tersebut akan terus dibagi dua hingga mencapai angka satu.
- Quick Sort: Beroperasi menggunakan prinsip *divide and conquer* (membagi dan menaklukkan). *Quick Sort* memilih salah satu elemen sebagai *pivot*, kemudian mempartisi *array* menjadi dua sub-himpunan, yakni elemen yang lebih kecil dari *pivot* dan elemen yang lebih besar.
- Merge Sort: Sama halnya dengan *Quick Sort*, algoritma ini memakai konsep *divide and conquer*. *Merge Sort* memecah kumpulan data menjadi bagian-bagian terkecil (tunggal), lalu menggabungkannya kembali (*merge*) secara perlahan dalam kondisi yang sudah terurut rapi.
- Bubble Sort: Ini adalah metode pengurutan paling dasar di mana elemen yang berdekatan dibandingkan satu sama lain, dan posisinya ditukar jika urutannya tidak sesuai. Proses perbandingan layaknya gelembung ini diulang hingga tidak ada lagi elemen yang perlu ditukar letaknya.

2.2 Alat dan Bahan

- 1) Perangkat keras (Hardware): Komputer atau Laptop.
- 2) Sistem Operasi (OS): Windows, macOS, atau distribusi Linux.
- 3) Perangkat Lunak (Software): Java Development Kit (JDK), serta Integrated Development Environment (IDE) seperti Eclipse.

2.3 Langkah Praktikum dan Kode Program

2.3.1 Kelas Shell Sort

```

package pekan8_2511532023;

public class ShellSort_2511532023 {
    public static void shellSort_2023(int[] A_2023) {
        int n_2023 = A_2023.length;
        int gap_2023 = n_2023/2;
        while (gap_2023 > 0) {
            for (int i_2023 = gap_2023; i_2023 < n_2023;
i_2023++) {
                int temp_2023 = A_2023[i_2023];
                int j_2023 = i_2023;
                while (j_2023 >= gap_2023 && A_2023[j_2023 -
gap_2023] > temp_2023) {
                    A_2023[j_2023] = A_2023[j_2023 -
gap_2023];
                    j_2023 = j_2023 - gap_2023;
                }
                A_2023[j_2023] = temp_2023;
            }
            gap_2023 = gap_2023/2;
        }
    }
    public static void main(String[] args) {
        int[] data_2023 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum          : ");
        printArray_2023(data_2023);

        shellSort_2023(data_2023);

        System.out.print("Sesudah (Shell Sort) : ");
        printArray_2023(data_2023);
    }
    public static void printArray_2023(int[] arr_2023) {
        for (int i_2023 : arr_2023) System.out.print(i_2023 + " ");
        System.out.println();
    }
}

```

Penjelasan Kode:

- `public class ShellSort_2511532023`: Deklarasi class utama program.
- `public static void shellSort_2023(int[] A_2023)`: Bertugas melakukan pengurutan. Fungsi ini menerima satu buah array bilangan bulat bernama `A_2023`.
- `int n_2023 = A_2023.length;`: Mengambil dan menyimpan total jumlah elemen (panjang) dari array `A_2023`.
- `int gap_2023 = n_2023/2;`: Menentukan jarak atau interval awal pengurutan, yaitu setengah dari total panjang array.

- `while (gap_2023 > 0)`: Perulangan utama yang akan terus berjalan selama nilai jarak (`gap_2023`) masih lebih besar dari 0.
- `for (int i_2023 = gap_2023; i_2023 < n_2023; i_2023++)`: Perulangan untuk menyalin elemen array, dimulai dari indeks sebesar nilai gap sampai elemen terakhir.
- `int temp_2023 = A_2023[i_2023];`: Menyimpan nilai elemen array saat ini ke dalam variabel sementara bernama `temp_2023` agar tidak hilang saat digeser.
- `int j_2023 = i_2023;`: Menyalin indeks posisi saat ini ke variabel `j_2023` untuk digunakan saat melakukan pengecekan mundur ke belakang.
- `while (j_2023 >= gap_2023 && A_2023[j_2023 - gap_2023] > temp_2023)`: Perulangan mundur untuk membandingkan elemen. Berjalan jika posisi indeks mencukupi dan elemen di sebelah kiri (sejauh jarak gap) nilainya lebih besar dari `temp_2023`.
- `A_2023[j_2023] = A_2023[j_2023 - gap_2023];`: Menggeser nilai elemen yang lebih besar di sebelah kiri ke posisi kanan.
- `j_2023 = j_2023 - gap_2023;`: Mengurangi nilai indeks `j_2023` sejauh jarak gap untuk terus memeriksa elemen-elemen sebelumnya di sebelah kiri.
- `A_2023[j_2023] = temp_2023;`: Menempatkan kembali nilai `temp_2023` ke posisi indeks `j_2023` yang tepat setelah proses pergeseran selesai.
- `gap_2023 = gap_2023/2;`: Memperkecil nilai jarak (gap) menjadi setengahnya di setiap akhir putaran untuk mempersempit jangkauan perbandingan.
- `public static void main(String[] args)`: Fungsi utama yang bertindak sebagai titik awal untuk menjalankan seluruh program Java.
- `int[] data_2023 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};`: Membuat sebuah array bilangan bulat acak bernama `data_2023` sebagai sampel data yang akan diurutkan.
- `System.out.print("Sebelum :");`: Menampilkan teks teks pembuka "Sebelum :" di layar konsol.
- `printArray_2023(data_2023);`: Memanggil fungsi cetak untuk menampilkan seluruh isi elemen array `data_2023` sebelum diurutkan.

- `shellSort_2023(data_2023);`:: Memanggil fungsi `shellSort_2023` untuk mengurutkan angka-angka di dalam array `data_2023`.
- `System.out.print("Sesudah (Shell Sort) :");`:: Menampilkan teks penanda "Sesudah (Shell Sort) :" di layar konsol.
- `printArray_2023(data_2023);`:: Memanggil fungsi cetak kembali untuk menampilkan isi elemen array `data_2023` yang sudah rapi terurut.
- `public static void printArray_2023(int[] arr_2023)`: Fungsi pembantu yang menerima sebuah array bernama `arr_2023` untuk mencetak isinya ke layar.
- `for (int i_2023 : arr_2023) System.out.print(i_2023 + " ");`: Perulangan `for-each` untuk mengambil setiap isi angka di dalam array satu per satu lalu mencetaknya ke samping dipisahkan spasi.
- `System.out.println();`:: Membuat baris baru di konsol setelah seluruh isi array selesai dicetak.

Output:

```

<terminated> ShellSort_2511532023 [Java Application] C:\Us
Sebelum           : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort) : 1 2 3 4 5 6 7 8 9 10
  
```

2.3.2 Kelas Quick Sort

```

package pekan8_2511532023;

public class QuickSort_2511532023 {
    static void swap_2023(int[] arr_2023, int i_2023, int j_2023) {
        int temp_2023 = arr_2023[i_2023];
        arr_2023[i_2023] = arr_2023[j_2023];
        arr_2023[j_2023] = temp_2023;
    }
    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_2023(int[] arr_2023, int low_2023, int high_2023) {
        int mid_2023 = low_2023 + (high_2023 - low_2023) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_2023[low_2023] > arr_2023[mid_2023]) {
            swap_2023(arr_2023, low_2023, mid_2023);
        }
        if (arr_2023[low_2023] > arr_2023[high_2023]) {
            swap_2023(arr_2023, low_2023, high_2023);
        }
        if (arr_2023[mid_2023] > arr_2023[high_2023]) {
  
```

```

        swap_2023(arr_2023, mid_2023, high_2023);
    }
    swap_2023(arr_2023, mid_2023, high_2023);
}
static int partition_2023(int[] arr_2023, int low_2023, int
high_2023) {
    // Panggil fungsi medianOfThree sebelum menentukan pivot
    medianOfThree_2023(arr_2023, low_2023, high_2023);

    int pivot_2023 = arr_2023[high_2023]; // Sekarang arr[high]
sudah berisi nilai median
    int i_2023 = (low_2023 - 1);

    for (int j_2023 = low_2023; j_2023 <= high_2023 - 1;
j_2023++) {
        // Jika elemen saat ini lebih kecil dari atau sama
dengan pivot
        if (arr_2023[j_2023] < pivot_2023) {
            // Increment indeks elemen yang lebih kecil
            i_2023++;
            swap_2023(arr_2023, i_2023, j_2023);
        }
    }
    swap_2023(arr_2023, i_2023 + 1, high_2023);
    return (i_2023 + 1);
}
static void quickSort_2023(int[] arr_2023, int low_2023, int
high_2023) {
    if (low_2023 < high_2023) {
        int pi_2023 = partition_2023(arr_2023, low_2023,
high_2023);

        quickSort_2023(arr_2023, low_2023, pi_2023 - 1);
        quickSort_2023(arr_2023, pi_2023 + 1, high_2023);
    }
}
public static void printArr_2023(int[] arr_2023) {
    for (int i_2023 = 0; i_2023 < arr_2023.length; i_2023++) {
        System.out.print(arr_2023[i_2023] + " ");
    }
    System.out.println();
}
public static void main(String[] args) {
    int[] arr_2023 = { 10, 7, 8, 9, 1, 5};
    int N_2023 = arr_2023.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2023(arr_2023);

    quickSort_2023(arr_2023, 0, N_2023 - 1);

    System.out.print("Data terurut quicksort: ");
    printArr_2023(arr_2023);
}
}

```

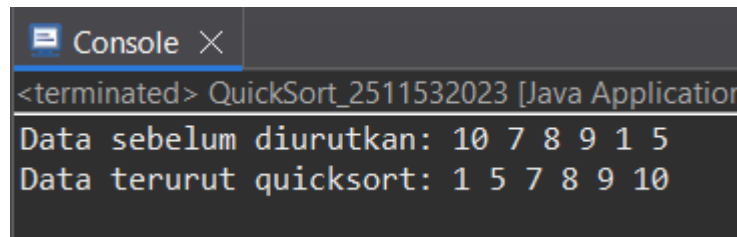
Penjelasan Kode:

- `public class QuickSort_2511532023 {` Membuat kelas utama untuk menjalankan algoritma Quick Sort.
- `static void swap_2023(int[] arr_2023, int i_2023, int j_2023) {` Fungsi pembantu yang bertugas menukar posisi dua elemen di dalam *array*.
- `int temp_2023 = arr_2023[i_2023]; ... (isi fungsi swap)` Menyimpan nilai pertama sementara, menyimpannya dengan nilai kedua, lalu menaruh nilai sementara ke posisi kedua.
- `static void medianOfThree_2023(int[] arr_2023, int low_2023, int high_2023) {` Fungsi ini mencari nilai tengah (median) dari tiga elemen untuk dijadikan patokan (*pivot*).
- `int mid_2023 = low_2023 + (high_2023 - low_2023) / 2;` Menghitung dan menentukan posisi indeks yang berada persis di tengah *array*.
- `if (arr_2023[low_2023] > arr_2023[mid_2023]) { swap_2023(...); }` (*dan blok if selanjutnya*) Mengurutkan elemen posisi awal, tengah, dan akhir dari yang terkecil ke terbesar.
- `swap_2023(arr_2023, mid_2023, high_2023);` Memindahkan elemen bernilai median ke posisi paling kanan agar siap digunakan sebagai *pivot*.
- `static int partition_2023(int[] arr_2023, int low_2023, int high_2023) {` Fungsi ini memisahkan angka ke kelompok kecil (kiri) dan kelompok besar (kanan) berdasarkan *pivot*.
- `medianOfThree_2023(arr_2023, low_2023, high_2023);` Memanggil fungsi pencari median agar *pivot* yang dipilih adalah nilai yang paling optimal.
- `int pivot_2023 = arr_2023[high_2023];` Menetapkan elemen di ujung paling kanan sebagai nilai patokan atau *pivot*.
- `int i_2023 = (low_2023 - 1);` Membuat variabel penunjuk (*i*) untuk melacak batas elemen-elemen yang bernilai lebih kecil dari *pivot*.
- `for (int j_2023 = low_2023; j_2023 <= high_2023 - 1; j_2023++) {` Perulangan untuk memeriksa nilai setiap elemen dari kiri sampai batas sebelum posisi *pivot*.

- `if (arr_2023[j_2023] < pivot_2023) {` Memeriksa apakah elemen yang sedang dicek nilainya lebih kecil dari *pivot*.
- `i_2023++;` `swap_2023(arr_2023, i_2023, j_2023);` Jika terbukti lebih kecil, batas penunjuk ditambah lalu elemen tersebut dilempar ke sisi kiri.
- `swap_2023(arr_2023, i_2023 + 1, high_2023);` Menaruh *pivot* ke tengah-tengah sebagai sekat pemisah antara kelompok angka kecil dan besar.
- `return (i_2023 + 1);` Mengembalikan informasi posisi indeks *pivot* yang baru untuk proses pemecahan selanjutnya.
- `static void quickSort_2023(int[] arr_2023, int low_2023, int high_2023)`
{ Ini adalah fungsi utama yang bersifat rekursif (memanggil dirinya sendiri) untuk mengurutkan data.
- `if (low_2023 < high_2023) {` Mencegah error dengan memastikan proses urut hanya berjalan jika data masih bisa dibelah minimal menjadi dua.
- `int pi_2023 = partition_2023(arr_2023, low_2023, high_2023);`
Menjalankan proses partisi dan menyimpan letak indeks sekat penengah (*pivot*) ke variabel `pi`.
- `quickSort_2023(arr_2023, low_2023, pi_2023 - 1);` Mengeksekusi ulang fungsi ini untuk mengurutkan sub-array khusus di sebelah kiri *pivot*.
- `quickSort_2023(arr_2023, pi_2023 + 1, high_2023);` Mengeksekusi ulang fungsi ini untuk mengurutkan sub-array khusus di sebelah kanan *pivot*.
- `public static void printArr_2023(int[] arr_2023) {` Fungsi pembantu yang khusus bertugas untuk mencetak isi *array* ke layar.
- `for (...)` `{ System.out.print(...)` `}` (*isi dari printArr*) Perulangan yang mencetak setiap angka di dalam *array* secara menyamping dan dipisah spasi.
- `public static void main(String[] args) {` Pusat eksekusi program di mana semua fungsi mulai dijalankan pertama kali.

- `int[] arr_2023 = { 10, 7, 8, 9, 1, 5};` Menyiapkan sekumpulan angka acak sebagai data simulasi yang akan diurutkan.
- `int N_2023 = arr_2023.length;` Menghitung total jumlah angka yang ada di dalam *array* simulasi tersebut.
- `quickSort_2023(arr_2023, 0, N_2023 - 1);` Memerintahkan program untuk mengurutkan seluruh data mulai dari indeks pertama hingga terakhir.

Output:



```
<terminated> QuickSort_2511532023 [Java Application]
Data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quicksort: 1 5 7 8 9 10
```

2.3.3 Kelas Merge Sort

```
package pekan8_2511532023;

public class MergeSort_2511532023 {
    void merge_2023(int arr_2023[], int l_2023, int m_2023, int
r_2023) {
        // Find sizes of two subarrays to be merged
        int n1_2023 = m_2023 - l_2023 + 1;
        int n2_2023 = r_2023 - m_2023;
        /* Create temp arrays */
        int L_2023[] = new int[n1_2023];
        int R_2023[] = new int[n2_2023];
        /* Copy data to temp arrays */
        for (int i_2023 = 0; i_2023 < n1_2023; ++i_2023)
            L_2023[i_2023] = arr_2023[l_2023 + i_2023];
        for (int j_2023 = 0; j_2023 < n2_2023; ++j_2023)
            R_2023[j_2023] = arr_2023[m_2023 + 1 + j_2023];
        int i_2023 = 0, j_2023 = 0;
        // Initial index of merged subarray array
        int k_2023 = l_2023;
        while (i_2023 < n1_2023 && j_2023 < n2_2023) {
            if (L_2023[i_2023] <= R_2023[j_2023]) {
                arr_2023[k_2023] = L_2023[i_2023];
                i_2023++;
            } else {
                arr_2023[k_2023] = R_2023[j_2023];
                j_2023++;
            }
            k_2023++;
        }
        /* Copy remaining elements of L[] if any */
        while (i_2023 < n1_2023) {
            arr_2023[k_2023] = L_2023[i_2023];
```

```

        i_2023++;
        k_2023++;
    }
    /* Copy remaining elements of R[] if any */
    while (j_2023 < n2_2023) {
        arr_2023[k_2023] = R_2023[j_2023];
        j_2023++;
        k_2023++;
    }
}

void sort_2023(int arr_2023[], int l_2023, int r_2023) {
    if (l_2023 < r_2023) {
        // Find the middle point
        int m_2023 = (l_2023 + r_2023) / 2;
        // Sort first and second halves
        sort_2023(arr_2023, l_2023, m_2023);
        sort_2023(arr_2023, m_2023 + 1, r_2023);
        // Merge the sorted halves
        merge_2023(arr_2023, l_2023, m_2023, r_2023);
    }
}

/* A utility function to print array of size n */
static void printArray_2023(int arr_2023[]) {
    int n_2023 = arr_2023.length;
    for (int i_2023 = 0; i_2023 < n_2023; ++i_2023)
        System.out.print(arr_2023[i_2023] + " ");
    System.out.println();
}

public static void main(String[] args) {
    int arr_2023[] = { 12, 11, 13, 5, 6, 7 };
    System.out.println("Sebelum terurut: ");
    printArray_2023(arr_2023);
    MergeSort_2511532023 ob_2023 = new MergeSort_2511532023();
    ob_2023.sort_2023(arr_2023, 0, arr_2023.length - 1);
    System.out.println("\nSesudah terurut menggunakan merge
Sort");
    printArray_2023(arr_2023);
}
}

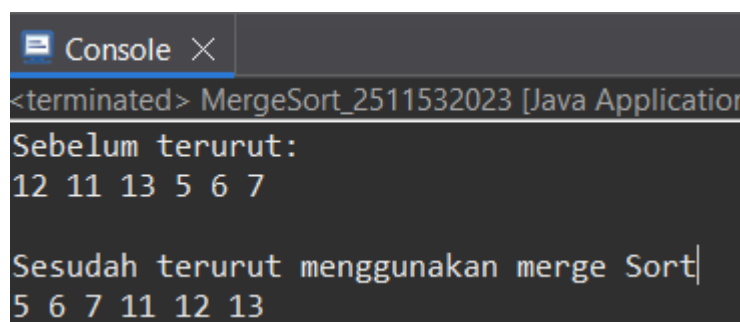
```

Penjelasan Kode:

- class MergeSort_2511532023: Kelas utama yang berisi kerangka algoritma.
- Fungsi merge_2023: Tugasnya menyatukan dua bagian *array* pecahan menjadi satu *array* yang sudah terurut.
 - n1_2023 & n2_2023: Mengukur seberapa panjang pecahan *array* kiri dan kanan.
 - L_2023[] & R_2023[]: Membuat *array* bayangan (wadah sementara) untuk menampung salinan nilai dari blok kiri (L) dan kanan (R).

- Dua buah for loop: Menyalin isi data dari *array* asli ke dalam wadah sementara L dan R.
- while ($i < n1 \ \&\& \ j < n2$): Proses adu nilai. Mengambil nilai terkecil antara wadah L dan R, lalu memasukkannya kembali ke *array* utama secara berurutan.
- Dua buah while terakhir: Tindakan "sapu bersih". Berfungsi memasukkan sisa elemen ke *array* utama jika salah satu wadah (L atau R) sudah habis duluan.
- Fungsi sort_2023: Berperan sebagai "Pembelah". Tugasnya memotong *array* menjadi potongan-potongan kecil.
 - $m_{2023} = (l_{2023} + r_{2023}) / 2$: Mencari titik tengah (ekuator) data.
 - sort_2023 (Rekursif): Memanggil dirinya sendiri untuk terus membelah *array* ke kiri dan ke kanan sampai tersisa satu elemen tunggal.
 - merge_2023: Setelah data hancur menjadi elemen tunggal, fungsi penggabung dipanggil untuk merakitnya kembali dari bawah ke atas dalam keadaan terurut.
- Fungsi printArray_2023: Fungsi pembantu yang hanya bertugas mencetak isi deretan angka ke layar konsol agar bisa dibaca manusia.
- Fungsi main: Area "Pusat Kendali" atau titik eksekusi jalannya program.
 - Menyiapkan sekumpulan angka acak (arr_2023).
 - Membuat objek kelas (ob_2023) karena fungsi pengurutannya tidak bersifat *static*.
 - Memerintahkan program untuk mengurutkan data dari indeks awal (0) hingga indeks paling akhir.
 - Mencetak data sebelum dan sesudah diurutkan.

Output:



```
<terminated> MergeSort_2511532023 [Java Application]
Sebelum terurut:
12 11 13 5 6 7

Sesudah terurut menggunakan merge Sort|
5 6 7 11 12 13
```

2.3.4 Kelas Bubble Sort GUI

```
package pekan8_2511532023;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.EventQueue;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511532023 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_2023;
    private JLabel[] labelArray_2023;
    private JButton stepButton_2023, resetButton_2023,
setButton_2023;
    private JTextField inputField_2023;
    private JPanel panelArray_2023;
    private JTextArea stepArea_2023;

    private int i_2023 = 1, j_2023;
    private boolean sorting_2023 = false;
    private int stepCount_2023 = 1;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511532023() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2023 = new JPanel(new FlowLayout());
        inputField_2023 = new JTextField(30);
        setButton_2023 = new JButton("Set Array");
        inputPanel_2023.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_2023.add(inputField_2023);
        inputPanel_2023.add(setButton_2023);
    }
}
```



```

// Panel array visual
panelArray_2023 = new JPanel();
panelArray_2023.setLayout(new FlowLayout());

// Panel kontrol
JPanel controlPanel_2023 = new JPanel();
stepButton_2023 = new JButton("Langkah Selanjutnya");
resetButton_2023 = new JButton("Reset");
stepButton_2023.setEnabled(false);
controlPanel_2023.add(stepButton_2023);
controlPanel_2023.add(resetButton_2023);

// Area teks untuk log langkah-langkah
stepArea_2023 = new JTextArea(8, 60);
stepArea_2023.setEditable(false);
stepArea_2023.setFont(new Font("Monospaced", Font.PLAIN,
14));

JScrollPane scrollPane = new JScrollPane(stepArea_2023);

// Tambahkan panel ke frame
add(inputPanel_2023, BorderLayout.NORTH);
add(panelArray_2023, BorderLayout.CENTER);
add(controlPanel_2023, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

// Event Set Array
setButton_2023.addActionListener(e ->
setArrayFromInput_2023());

// Event Langkah Selanjutnya
stepButton_2023.addActionListener(e ->
performStep_2023());

// Event Reset
resetButton_2023.addActionListener(e -> reset_2023()); }

private void setArrayFromInput_2023() {
    String text_2023 = inputField_2023.getText().trim();
    if (text_2023.isEmpty()) return;
    String[] parts = text_2023.split(",");
    array_2023 = new int[parts.length];
    try {
        for (int k_2023 = 0; k_2023 < parts.length;
k_2023++) {
            array_2023[k_2023] =
Integer.parseInt(parts[k_2023].trim()); }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka " + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2023 = 0;
        j_2023 = 0;
        stepCount_2023 = 1;
        sorting_2023 = true;
        stepButton_2023.setEnabled(true);
        stepArea_2023.setText("");
        panelArray_2023.removeAll();

```

```

        labelArray_2023 = new JLabel[array_2023.length];
        for (int k_2023 = 0; k_2023 < array_2023.length;
k_2023++) {
            labelArray_2023[k_2023] = new
JLabel(String.valueOf(array_2023[k_2023]));
            labelArray_2023[k_2023].setFont(new Font("Arial",
Font.BOLD, 24));
            labelArray_2023[k_2023].setOpaque(true);

labelArray_2023[k_2023].setBackground(Color.WHITE);

labelArray_2023[k_2023].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
            labelArray_2023[k_2023].setPreferredSize(new
Dimension(50, 50));

labelArray_2023[k_2023].setHorizontalAlignment(SwingConstants.CENT
ER);
            panelArray_2023.add(labelArray_2023[k_2023]);
        }
        panelArray_2023.revalidate();
        panelArray_2023.repaint(); }

private void performStep_2023() {
    if (!sorting_2023 || i_2023 >= array_2023.length - 1)
{
        sorting_2023 = false;
        stepButton_2023.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        return; }
    resetHighlights_2023();
    StringBuilder stepLog_2023 = new StringBuilder();
    labelArray_2023[j_2023].setBackground(Color.CYAN);
    labelArray_2023[j_2023 + 1].setBackground(Color.CYAN);
    if (array_2023[j_2023] > array_2023[j_2023 + 1]) {
        // Swap
        int temp_2023 = array_2023[j_2023];
        array_2023[j_2023] = array_2023[j_2023 + 1];
        array_2023[j_2023 + 1] = temp_2023;
        labelArray_2023[j_2023].setBackground(Color.RED);
        labelArray_2023[j_2023 +
1].setBackground(Color.RED);
        stepLog_2023.append("Langkah
").append(stepCount_2023).append(": Menukar elemen ke-")
            .append(j_2023).append("
").append(array_2023[j_2023 + 1]).append(" dengan ke-")
            .append(j_2023 + 1).append("
").append(array_2023[j_2023]).append(")\n");
        } else {
            stepLog_2023.append("Langkah
").append(stepCount_2023).append(": Tidak ada pertukaran antara
ke-")
                .append(j_2023).append(" dan ke-")
                .append(j_2023 + 1).append(")\n"); }
        stepLog_2023.append("Hasil:
").append(arrayToString_2023(array_2023)).append("\n\n");
        stepArea_2023.append(stepLog_2023.toString());
        updateLabels_2023();

```

```

        j_2023++;
        if (j_2023 >= array_2023.length - i_2023 - 1) {
            j_2023 = 0;
            i_2023++; }
        stepCount_2023++;
        if (i_2023 >= array_2023.length - 1) {
            sorting_2023 = false;
            stepButton_2023.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }

        private void updateLabels_2023() {
            for (int k_2023 = 0; k_2023 < array_2023.length;
k_2023++) {
                labelArray_2023[k_2023].setText(String.valueOf(array_2023[k_2023])
);
            }
        }
        private void resetHighlights_2023() {
            for (JLabel label : labelArray_2023) {
                label.setBackground(Color.WHITE);
            }
        }
        private void reset_2023() {
            inputField_2023.setText("");
            panelArray_2023.removeAll();
            panelArray_2023.revalidate();
            panelArray_2023.repaint();
            stepArea_2023.setText("");
            stepButton_2023.setEnabled(false);
            sorting_2023 = false;
            i_2023 = 0;
            j_2023 = 0;
            stepCount_2023 = 1;
        }

        private String arrayToString_2023(int[] arr_2023) {
            StringBuilder sb_2023 = new StringBuilder();
            for (int k_2023 = 0; k_2023 < arr_2023.length;
k_2023++) {
                sb_2023.append(arr_2023[k_2023]);
                if (k_2023 < arr_2023.length - 1)
sb_2023.append(", ");
            }
            return sb_2023.toString();
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> {
                BubbleSortGUI_2511532023 gui_2023 = new
BubbleSortGUI_2511532023();
                gui_2023.setVisible(true);
            });
        }
    }
}

```

Penjelasan Kode:

- `public class BubbleSortGUI_2511532023 extends JFrame`: Mendeklarasikan kelas utama yang mewarisi sifat `JFrame` untuk membuat jendela antarmuka aplikasi (GUI) di layar komputer.
- Deklarasi Variabel Global (`array`, `JLabel[]`, `JButton`, dll): Menyiapkan variabel untuk menampung komponen tombol, area teks, kotak angka, serta variabel pelacak indeks (`i`, `j`) untuk mengatur jalannya pengurutan.
- Konstruktor `BubbleSortGUI_2511532023()`: Bertugas merakit seluruh tata letak antarmuka, seperti memasang kolom *input*, menyiapkan panel, dan mengaitkan tombol dengan perintahnya saat aplikasi baru pertama kali dibuka.
- Fungsi `setArrayFromInput_2023()`: Membaca teks angka berbatas koma yang diketik pengguna, memecahnya menjadi data *array* sungguhan, lalu mencetak kotak-kotak visual angka tersebut ke tengah layar.
- Fungsi `performStep_2023()`: Menjalankan satu tahap dari logika *Bubble Sort* di mana ia membandingkan dua kotak angka yang berdekatan setiap kali tombol "Langkah Selanjutnya" ditekan.
- Blok `if-else` dalam `performStep_2023`: Mewarnai kotak menjadi *Cyan* saat angka sedang dibandingkan, lalu mengubahnya menjadi *Merah* dan menukar posisinya jika angka sebelah kiri lebih besar dari angka sebelah kanan.
- Fungsi `updateLabels_2023()`: Bertugas memperbarui tampilan teks di dalam kotak-kotak GUI agar posisinya selalu sama dan sinkron dengan susunan data *array* terbaru.
- Fungsi `resetHighlights_2023()`: Mengembalikan warna latar belakang seluruh kotak angka menjadi putih polos agar layar bersih kembali untuk langkah berikutnya.
- Fungsi `reset_2023()`: Membersihkan seluruh data masukan, menghapus visual kotak angka, dan mengosongkan riwayat log sehingga program siap dipakai ulang dari awal.

- Fungsi `arrayToString_2023()`: Menyusun deretan angka *array* menjadi baris teks terbatas koma agar mudah dibaca saat dicetak ke dalam kotak riwayat log langkah-langkah.
- Fungsi `main & SwingUtilities.invokeLater`: Titik mulai program yang bertugas memunculkan jendela aplikasi GUI ke layar pengguna dengan aman dan stabil.

Output:



2.3.5 Kelas Merge Sort GUI

```
package pekan7_2511532023;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.EventQueue;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511532023 extends JFrame {
```

```

    private static final long serialVersionUID = 1L;
    private int[] array_2023;
    private JLabel[] labelArray_2023;
    private JButton stepButton_2023, resetButton_2023,
setButton_2023;
    private JTextField inputField_2023;
    private JPanel panelArray_2023;
    private JTextArea stepArea_2023;

    private int i_2023 = 1, j_2023;
    private boolean sorting_2023 = false;
    private int stepCount_2023 = 1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532023() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2023 = new JPanel(new FlowLayout());
        inputField_2023 = new JTextField(30);
        setButton_2023 = new JButton("Set Array");
        inputPanel_2023.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_2023.add(inputField_2023);
        inputPanel_2023.add(setButton_2023);

        // Panel array visual
        panelArray_2023 = new JPanel();
        panelArray_2023.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel = new JPanel();
        stepButton_2023 = new JButton("Langkah Selanjutnya");
        resetButton_2023 = new JButton("Reset");
        stepButton_2023.setEnabled(false);
        controlPanel.add(stepButton_2023);
        controlPanel.add(resetButton_2023);

        // Area teks untuk log langkah-langkah
        stepArea_2023 = new JTextArea(8, 60);
        stepArea_2023.setEditable(false);
        stepArea_2023.setFont(new Font("Monospaced", Font.PLAIN,
14));
        JScrollPane scrollPane = new JScrollPane(stepArea_2023);

        // Tambahkan panel ke frame
        add(inputPanel_2023, BorderLayout.NORTH);
        add(panelArray_2023, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        // Event Set Array

```

```

        setButton_2023.addActionListener(e ->
setArrayFromInput_2023());

        // Event Langkah Selanjutnya
        stepButton_2023.addActionListener(e ->
performStep_2023());

        // Event Reset
        resetButton_2023.addActionListener(e -> reset_2023()); }

private void setArrayFromInput_2023() {
    String text_2023 = inputField_2023.getText().trim();
    if (text_2023.isEmpty()) return;
    String[] parts = text_2023.split(",");
    array_2023 = new int[parts.length];
    try {
        for (int k_2023 = 0; k_2023 < parts.length;
k_2023++) {
            array_2023[k_2023] =
Integer.parseInt(parts[k_2023].trim()); }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka yang dipisahkan " + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2023 = 1;
        stepCount_2023 = 1;
        sorting_2023 = true;
        stepButton_2023.setEnabled(true);
        stepArea_2023.setText("");
        panelArray_2023.removeAll();
        labelArray_2023 = new JLabel[array_2023.length];
        for (int k_2023 = 0; k_2023 < array_2023.length;
k_2023++) {
            labelArray_2023[k_2023] = new
JLabel(String.valueOf(array_2023[k_2023]));
            labelArray_2023[k_2023].setFont(new Font("Arial",
Font.BOLD, 24));

            labelArray_2023[k_2023].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
            labelArray_2023[k_2023].setPreferredSize(new
Dimension(50, 50));

            labelArray_2023[k_2023].setHorizontalAlignment(SwingConstants.CENT
ER);

            panelArray_2023.add(labelArray_2023[k_2023]);
        }
        panelArray_2023.revalidate();
        panelArray_2023.repaint(); }

private void performStep_2023() {
    if (i_2023 < array_2023.length && sorting_2023) {
        int key_2023 = array_2023[i_2023];
        j_2023 = i_2023 - 1;

        StringBuilder stepLog_2023 = new StringBuilder();

```

```

        stepLog_2023.append("Langkah
").append(stepCount_2023).append(": Memasukkan
").append(key_2023).append("\n");

        while (j_2023 >= 0 && array_2023[j_2023] >
key_2023) {
            array_2023[j_2023 + 1] = array_2023[j_2023];
            j_2023--;
        }
        array_2023[j_2023 + 1] = key_2023;

        updateLabels_2023();
        stepLog_2023.append("Hasil:
").append(arrayToString_2023(array_2023)).append("\n\n");
        stepArea_2023.append(stepLog_2023.toString());

        i_2023++;
        stepCount_2023++;

        if (i_2023 == array_2023.length) {
            sorting_2023 = false;
            stepButton_2023.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }
    }

    private void updateLabels_2023() {
        for (int k_2023 = 0; k_2023 < array_2023.length;
k_2023++) {
            labelArray_2023[k_2023].setText(String.valueOf(array_2023[k_2023])
);
        }
    }

    private void reset_2023() {
        inputField_2023.setText("");
        panelArray_2023.removeAll();
        panelArray_2023.revalidate();
        panelArray_2023.repaint();
        stepArea_2023.setText("");
        stepButton_2023.setEnabled(false);
        sorting_2023 = false;
        i_2023 = 1;
        stepCount_2023 = 1;
    }

    private String arrayToString_2023(int[] arr_2023) {
        StringBuilder sb_2023 = new StringBuilder();
        for (int k_2023 = 0; k_2023 < arr_2023.length;
k_2023++) {
            sb_2023.append(arr_2023[k_2023]);
            if (k_2023 < arr_2023.length - 1)
                sb_2023.append(", ");
        }
        return sb_2023.toString();
    }

    public static void main(String[] args) {

```



```

        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511532023 gui_2023 = new
InsertionSortGUI_2511532023();
            gui_2023.setVisible(true);
        });
    }
}

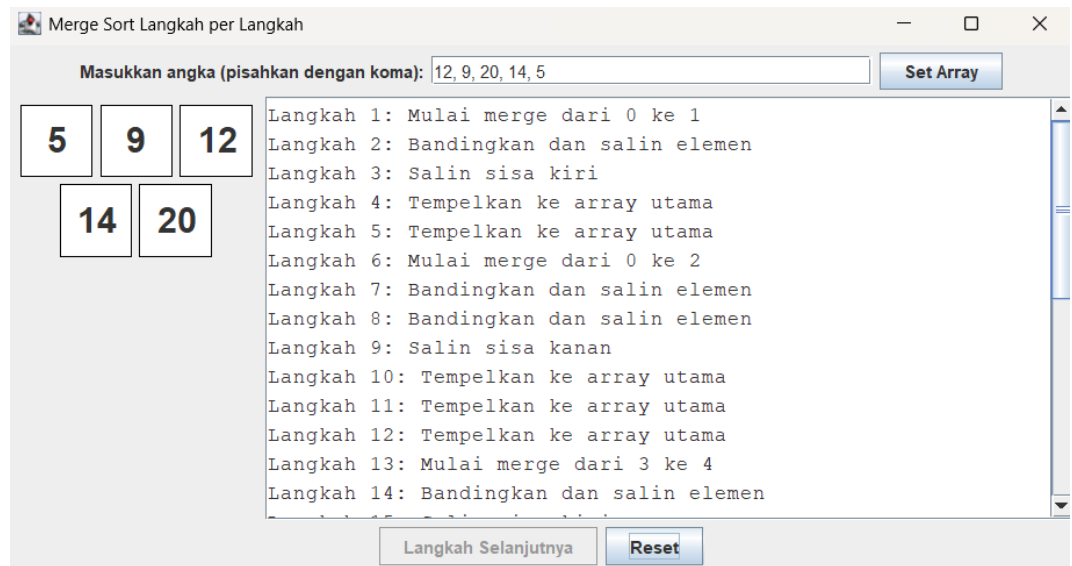
```

Penjelasan Kode:

- `public class MergeSortGUI_2511532023 extends JFrame`: Mendeklarasikan kelas utama yang mewarisi komponen `JFrame` untuk membangun jendela antarmuka visual program.
- Deklarasi Variabel Global (`Queue`, `array`, `JLabel[]`, `dll`): Menyiapkan variabel komponen layar, wadah *array* angka, serta struktur data antrean (`Queue`) khusus untuk menyimpan urutan langkah pengurutan.
- Konstruktor `MergeSortGUI_2511532023()`: Bertugas merakit kerangka tata letak jendela, memasang panel *input*, menyiapkan area log riwayat, dan menyambungkan tombol dengan perintahnya.
- Fungsi `setArrayFromInput_2023()`: Mengambil teks angka masukan pengguna, mengubahnya menjadi kotak-kotak visual di layar, lalu memicu proses kalkulasi langkah pembelahan data.
- Fungsi `generateMergeSteps_2023()`: Membelah jangkauan *array* secara rekursif di balik layar dan mencatat semua indeks potongannya ke dalam antrean (`Queue`) agar GUI bisa memprosesnya langkah demi langkah secara berurutan.
- Fungsi `performStep_2023()`: Mengeksekusi antrean penggabungan data satu per satu, mengubah kotak menjadi warna *Cyan* saat dibandingkan, dan warna *Hijau* saat nilai sukses digabung (*di-merge*) kembali ke barisan aslinya.
- Fungsi `updateLabels_2023()`: Memperbarui teks angka di dalam kotak-kotak visual agar posisinya selalu sinkron dengan perubahan yang terjadi pada data *array* di memori.
- Fungsi `resetHighlights_2023()`: Mengembalikan warna latar seluruh kotak angka menjadi putih polos setelah proses pemindahan atau perbandingan di satu tahap selesai.

- Fungsi `reset_2023()`: Menghapus seluruh kotak visual, mengosongkan antrean, dan membersihkan teks riwayat log sehingga program bersih dan siap menerima data baru.
- Fungsi `arrayToString_2023()`: Mengonversi susunan angka dari *array* menjadi format kalimat teks berbatas koma agar rapi saat dicetak ke kotak log langkah-langkah.
- Fungsi `main & SwingUtilities.invokeLater()`: Titik mula berjalannya program yang bertugas meluncurkan dan menampilkan jendela aplikasi GUI ke layar dengan aman dan stabil.

Output:



BAB III

PENUTUP

3.1 Kesimpulan

Dari keseluruhan rangkaian pengerjaan kode dan observasi, dapat ditarik konklusi bahwa setiap algoritma pengurutan mempunyai skenario penerapan terbaiknya sendiri. Algoritma dasar seperti *Bubble Sort* amat mudah diimplementasikan, tetapi sangat membebani komputasi ketika datanya melimpah. Di lain sisi, metode mutakhir dengan paradigma *divide and conquer* seperti *Quick Sort* dan *Merge Sort* menuntaskan tugas jauh lebih kilat untuk skala besar, meski sintaks yang dirakit jauh lebih rumit.

Pemanfaatan representasi antarmuka visual (GUI) menggunakan pustaka Swing memberikan dimensi baru dalam memahami interaksi komputasional. Hal-hal yang tadinya abstrak, seperti pergeseran *pointer* dan pencadangan *array* sementara (*temporary variables*), berubah menjadi pergerakan kotak warna-warni yang runut. Kombinasi teori murni dengan visualisasi praktik ini sukses membuktikan esensi kinerja algoritma di balik layar sebuah perangkat lunak.

3.2 Saran

Untuk penyempurnaan program di waktu mendatang, disarankan untuk menyuntikkan prosedur penanganan eksepsi (*Exception Handling/Try-Catch*) tingkat lanjut pada antarmuka GUI, khususnya guna menolak masukan *string* nakal (karakter alfabet) dari pengguna agar *software* tidak tiba-tiba mogok (*crash*).

Selain itu, efisiensi konsumsi memori harus ditinjau ulang apabila kelak berencana me-render ratusan blok elemen *array* secara visual. Pada tingkatan operasional praktikum, membiasakan diri untuk memanfaatkan sistem kontrol versi (*version control*) akan sangat berguna untuk melacak perubahan skrip dari waktu ke waktu secara presisi, sekaligus meminimalkan ancaman hilangnya kode akibat disfungsi pada utilitas penyimpanan atau rintangan kerusakan *hardware*.

DAFTAR PUSTAKA

[1] D. Sundari, "Metode Sorting dalam Pemrograman Java," Laporan Praktikum Pemograman JAVA, Politeknik Negeri Bandung, Bandung, Indonesia, 2010.

[Daring]. Tersedia: <https://id.scribd.com/doc/187440738/LAPORAN-praktikum-Java-Sorting> [Diakses: 4 Jun. 2026].

[2] D. Zulfikarrea, "Praktikum Algoritma & Struktur Dasar Sorting (Pengurutan)," Laporan Praktikum, Universitas Amikom Purwokerto, Purwokerto, Indonesia, 2024. [Daring]. Tersedia: <https://id.scribd.com/document/1011892038/Praktikum-Algoritma-Struktur-Dasar-Sorting-Pengurutan> [Diakses: 5 Jun. 2026].

[3] Anonim, "Algoritma Sorting: Insertion dan Selection," Laporan Praktikum, 2021. [Daring]. Tersedia: <https://id.scribd.com/document/541575190/LAPORAN-SORTING> [Diakses: 5 Jun. 2026].

TUGAS 8 PRAKTIKUM STRUKTUR DATA

Soal

Mahasiswa diminta mengimplementasikan algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

Jawab

1. Kelas Lagu_2511532023

Kelas ini dibuat untuk menampung atribut judul, penyanyi, dan durasi.

```
package pekan8_2511532023;

public class Lagu_2511532023 {
    String judul_2023;
    String penyanyi_2023;
    int durasi_2023;

    public Lagu_2511532023(String judul_2023, String
    penyanyi_2023, int durasi_2023) {
        this.judul_2023 = judul_2023;
        this.penyanyi_2023 = penyanyi_2023;
        this.durasi_2023 = durasi_2023;
    }
}
```

Penjelasan Kode:

A. Deklarasi

- package pekan8_2511532023; Mendefinisikan nama folder atau lokasi tempat file Java ini disimpan agar program terorganisir dan tidak bentrok dengan *project* lain.
- public class Lagu_2511532023 { ... } Membuat sebuah "cetakan" (blueprint) utama yang bersifat publik dan bisa diakses dari luar untuk menciptakan objek lagu.

B. Pembuatan Variabel

- String judul_2023; hingga int durasi_2023; Mendeklarasikan atribut (variabel kelas) untuk menyediakan ruang penyimpanan data berupa teks dan angka bulat yang dimiliki oleh setiap lagu.

C. Konstruktor

- `public Lagu_2511532023(...) { this... }` Merupakan Constructor (metode khusus pembuat objek) yang otomatis berjalan saat lagu baru dibuat, sekaligus menggunakan perintah `this` untuk menyimpan data masukan (parameter) ke dalam atribut asli milik kelas.

2. Kelas Sorting_2511532023

```
package pekan8_2511532023;

import java.util.Scanner;

public class Sorting_2511532023 {
    // Array dataLagu dengan kapasitas maks 20 lagu
    Lagu_2511532023[] dataLagu_2023 = new Lagu_2511532023[20];
    int jumlahLagu_2023 = 0;

    // Method untuk mengisi data awal, min 7 lagu
    public void inputData_2023() {
        dataLagu_2023[0] = new Lagu_2511532023("Mio Cristo Piange
Diamanti", "Artist A", 270);
        dataLagu_2023[1] = new Lagu_2511532023("La Rumba Del Perdon",
"Artist B", 252);
        dataLagu_2023[2] = new Lagu_2511532023("La Perla", "Artist C",
196);
        dataLagu_2023[3] = new Lagu_2511532023("A Sky Full of Stars",
"Coldplay", 268);
        dataLagu_2023[4] = new Lagu_2511532023("Viva La Vida",
"Coldplay", 242);
        dataLagu_2023[5] = new Lagu_2511532023("Bohemian Rhapsody",
"Queen", 354);
        dataLagu_2023[6] = new Lagu_2511532023("Yellow", "Coldplay",
266);

        jumlahLagu_2023 = 7;
    }

    // Method menampilkan data sebelum dan sesudah sorting
    public void tampilData_2023() {
        for (int i = 0; i < jumlahLagu_2023; i++) {
            System.out.println((i + 1) + ". " +
dataLagu_2023[i].judul_2023 + " - " + dataLagu_2023[i].durasi_2023 + "
detik");
        }
    }

    // Algoritma 1: Shell Sort (Judul A-Z)
    public void shellSort_2023() {
        int n_2023 = jumlahLagu_2023;

        for (int gap_2023 = n_2023 / 2; gap_2023 > 0; gap_2023 =
gap_2023 / 2) {
            for (int i = gap_2023; i < n_2023; i++) {
                Lagu_2511532023 temp_2023 = dataLagu_2023[i];
```

```

        int j = i;

        // Menggeser elemen yang lebih besar ke kanan
        while (j >= gap_2023 && dataLagu_2023[j -
gap_2023].judul_2023.compareToIgnoreCase(temp_2023.judul_2023) > 0) {
            dataLagu_2023[j] = dataLagu_2023[j - gap_2023];
            j = j - gap_2023;
        }

        // Meletakkan elemen pada posisi yang benar
        dataLagu_2023[j] = temp_2023;
    }
}

// Algoritma 2: Quick Sort (Durasi Ascending)
public void quickSort_2023(int low_2023, int high_2023) {
    if (low_2023 < high_2023) {
        int pi_2023 = partition_2023(low_2023, high_2023);

        // Rekursif mengurutkan bagian kiri dan kanan pivot
        quickSort_2023(low_2023, pi_2023 - 1);
        quickSort_2023(pi_2023 + 1, high_2023);
    }
}

public int partition_2023(int low_2023, int high_2023) {
    int pivot = dataLagu_2023[high_2023].durasi_2023; // pivot di
elemen paling kanan
    int i = (low_2023 - 1); // index untuk elemen yang lebih kecil

    for (int j = low_2023; j < high_2023; j++) {
        // Jika durasi saat ini < pivot
        if (dataLagu_2023[j].durasi_2023 < pivot) {
            i++;
            // Swap dataLagu[i] dengan dataLagu[j]
            Lagu_2511532023 temp = dataLagu_2023[i];
            dataLagu_2023[i] = dataLagu_2023[j];
            dataLagu_2023[j] = temp;
        }
    }
    Lagu_2511532023 temp = dataLagu_2023[i + 1];
    dataLagu_2023[i + 1] = dataLagu_2023[high_2023];
    dataLagu_2023[high_2023] = temp;

    return i + 1; // kembalikan posisi pivot
}

// Algoritma 3: Merge Sort (Judul A-Z)
public void mergeSort_2023(int left_2023, int right_2023) {
    if (left_2023 < right_2023) {
        int mid_2023 = left_2023 + (right_2023 - left_2023) / 2;

        mergeSort_2023(left_2023, mid_2023);
        mergeSort_2023(mid_2023 + 1, right_2023);

        merge_2023(left_2023, mid_2023, right_2023);
    }
}

```

```

    }
}

public void merge_2023(int left_2023, int mid_2023, int right_2023)
{
    int n1_2023 = mid_2023 - left_2023 + 1;
    int n2_2023 = right_2023 - mid_2023;

    Lagu_2511532023[] L = new Lagu_2511532023[n1_2023];
    Lagu_2511532023[] R = new Lagu_2511532023[n2_2023];

    for (int i_2023 = 0; i_2023 < n1_2023; i_2023++) {
        L[i_2023] = dataLagu_2023[left_2023 + i_2023];
    }
    for (int j_2023 = 0; j_2023 < n2_2023; j_2023++) {
        R[j_2023] = dataLagu_2023[mid_2023 + 1 + j_2023];
    }

    int i_2023 = 0, j_2023 = 0;
    int k_2023 = left_2023;

    while (i_2023 < n1_2023 && j_2023 < n2_2023) {
        if
(L[i_2023].judul_2023.compareToIgnoreCase(R[j_2023].judul_2023) <= 0) {
            dataLagu_2023[k_2023] = L[i_2023];
            i_2023++;
        } else {
            dataLagu_2023[k_2023] = R[j_2023];
            j_2023++;
        }
        k_2023++;
    }

    while (i_2023 < n1_2023) {
        dataLagu_2023[k_2023] = L[i_2023];
        i_2023++;
        k_2023++;
    }

    while (j_2023 < n2_2023) {
        dataLagu_2023[k_2023] = R[j_2023];
        j_2023++;
        k_2023++;
    }
}

// Main Method
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Sorting_2511532023 program_2023 = new Sorting_2511532023();

    while (true) {
        // Memasukkan ulang data awal setiap kali loop berjalan
        program_2023.inputData_2023();

        System.out.println("\n=== Sorting Playlist NIM: 2511532023
===");
    }
}

```



```

        System.out.print("Pilih Algoritma (1=Shell, 2=Quick,
3=Merge): ");
        int pilihan_2023 = sc.nextInt();

        // Jika pilihan valid (1, 2, atau 3), tampilkan data awal
        if (pilihan_2023 >= 1 && pilihan_2023 <= 3) {
            System.out.println("\nData Sebelum Sorting:");
            program_2023.tampilData_2023();
        }

        // Logika pemilihan algoritma
        if (pilihan_2023 == 1) {
            program_2023.shellSort_2023();
            System.out.println("\nData Setelah Shell Sort (Judul A-
Z):");
            program_2023.tampilData_2023();
        } else if (pilihan_2023 == 2) {
            program_2023.quickSort_2023(0,
program_2023.jumlahLagu_2023 - 1);
            System.out.println("\nData Setelah Quick Sort (Durasi
Asc):");
            program_2023.tampilData_2023();
        } else if (pilihan_2023 == 3) {
            program_2023.mergeSort_2023(0,
program_2023.jumlahLagu_2023 - 1);
            System.out.println("\nData Setelah Merge Sort (Judul A-
Z):");
            program_2023.tampilData_2023();
        } else {
            System.out.println("\nPilihan tidak valid! Silakan pilih
1, 2, atau 3.");
        }
    }
}
}

```

Analisis Kode:

A. Package & Import

```
package pekan8_2511532023;
```

```
import java.util.Scanner;
```

Mengatur lokasi folder penyimpanan file Java ini dan memanggil alat bantu Scanner agar program bisa membaca ketikan angka dari pengguna lewat *keyboard*.

B. Deklarasi & Atribut

```
public class Sorting_2511532023 {
```

```
    Lagu_2511532023[] dataLagu_2023 = new Lagu_2511532023[20];
```

```
    int jumlahLagu_2023 = 0;
```

Membuat kelas utama yang menyiapkan sebuah *array* berkapasitas maksimal 20 ruang untuk menampung data lagu, beserta variabel untuk mencatat jumlah lagu yang terisi.

C. Method Pengisian Data

```
public void inputData_2023() {  
    dataLagu_2023[0] = new Lagu_2511532023("Mio Cristo Piange  
Diamanti", "Artist A", 270);  
  
    .....  
    jumlahLagu_2023 = 7;  
}
```

Berfungsi untuk mendaftarkan 7 data lagu awal (judul, penyanyi, durasi) secara otomatis ke dalam *array* setiap kali dipanggil.

D. Method Penampilan Data

```
public void tampilData_2023() {  
    for (int i = 0; i < jumlahLagu_2023; i++) {  
        System.out.println((i + 1) + ". " + dataLagu_2023[i].judul_2023 + " -  
" + dataLagu_2023[i].durasi_2023 + " detik");  
    }  
}
```

Bertugas membaca isi *array* satu per satu menggunakan perulangan lalu mencetak nomor, judul lagu, beserta durasinya ke layar.

E. Algoritma 1: Shell Sort

```
public void shellSort_2023() {  
    int n_2023 = jumlahLagu_2023;  
    for (int gap_2023 = n_2023 / 2; gap_2023 > 0; gap_2023 = gap_2023 /  
2) {  
        .....  
    }  
}
```

Mengurutkan lagu berdasarkan judul sesuai abjad (A-Z) dengan cara membandingkan dan menggeser posisi elemen-elemen yang terpisahkan oleh jarak (*gap*) tertentu.

F. Algoritma 2: Quick Sort

```
public void quickSort_2023(int low_2023, int high_2023) { ... }  
public int partition_2023(int low_2023, int high_2023) { ... }
```

Mengurutkan lagu berdasarkan durasi terpendek ke terpanjang dengan memutar-mutar posisi data mengelilingi satu titik acuan (*pivot*) hingga semua posisinya terurut benar.

G. Algoritma 3: Merge Sort

```
public void mergeSort_2023(int left_2023, int right_2023) { ... }  
public void merge_2023(int left_2023, int mid_2023, int right_2023) { ... }
```

Mengurutkan lagu berdasarkan judul abjad (A-Z) dengan cara membelah data menjadi bagian-bagian sangat kecil, lalu menjahitnya/menggabungkannya kembali (*merge*) dalam keadaan sudah rapi.

H. Main Method

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    Sorting_2511532023 program_2023 = new Sorting_2511532023();  
  
    while (true) {  
        .....  
    }  
}  
}
```

Merupakan jantung berjalannya program yang berisi perulangan tanpa batas (*while(true)*) untuk secara otomatis mereset susunan lagu, memunculkan menu algoritma, dan mengeksekusi metode pilihan pengguna.

Output:

- Pilih Algoritma 1

```
Console X
Sorting_2511532023 [Java Application] C:\Users\ACER\p2\pool\p

=== Sorting Playlist NIM: 2511532023 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 1

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. A Sky Full of Stars - 268 detik
5. Viva La Vida - 242 detik
6. Bohemian Rhapsody - 354 detik
7. Yellow - 266 detik

Data Setelah Shell Sort (Judul A-Z):
1. A Sky Full of Stars - 268 detik
2. Bohemian Rhapsody - 354 detik
3. La Perla - 196 detik
4. La Rumba Del Perdon - 252 detik
5. Mio Cristo Piange Diamanti - 270 detik
6. Viva La Vida - 242 detik
7. Yellow - 266 detik
```

- Pilih Algoritma 2

```
Console X
Sorting_2511532023 [Java Application] C:\Users\ACER\p2\pool\p
=== Sorting Playlist NIM: 2511532023 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. A Sky Full of Stars - 268 detik
5. Viva La Vida - 242 detik
6. Bohemian Rhapsody - 354 detik
7. Yellow - 266 detik

Data Setelah Quick Sort (Durasi Asc):
1. La Perla - 196 detik
2. Viva La Vida - 242 detik
3. La Rumba Del Perdon - 252 detik
4. Yellow - 266 detik
5. A Sky Full of Stars - 268 detik
6. Mio Cristo Piange Diamanti - 270 detik
7. Bohemian Rhapsody - 354 detik
```

- Pilih Algoritma 3

```
Console X
Sorting_2511532023 [Java Application] C:\Users\ACER\p2\pool\plugins\

=== Sorting Playlist NIM: 2511532023 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 3

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. A Sky Full of Stars - 268 detik
5. Viva La Vida - 242 detik
6. Bohemian Rhapsody - 354 detik
7. Yellow - 266 detik

Data Setelah Merge Sort (Judul A-Z):
1. A Sky Full of Stars - 268 detik
2. Bohemian Rhapsody - 354 detik
3. La Perla - 196 detik
4. La Rumba Del Perdon - 252 detik
5. Mio Cristo Piange Diamanti - 270 detik
6. Viva La Vida - 242 detik
7. Yellow - 266 detik
```

- Jika pilih selain angka 1, 2, dan 3

```
Console X
Sorting_2511532023 [Java Application] C:\Users\ACER\p2\pool\plugins\

=== Sorting Playlist NIM: 2511532023 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 4

Pilihan tidak valid! Silakan pilih 1, 2, atau 3.
```